1

Desenvolvimento de Software 1 · Semana 1

Introdução à gestão da engenharia de software

A engenharia de software é uma disciplina sobre entregar e manter software, não apenas escrevê-lo.

- A maioria dos problemas de projeto vem de falhas de comunicação e de planejamento.
- Os modelos permitem que as equipes discutam e verifiquem um sistema antes de construí-lo.

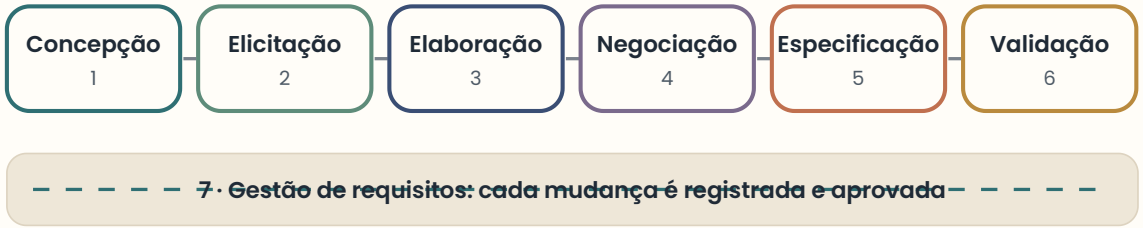
> **Pense assim.** Construir uma casa exige plantas antes dos tijolos. O software precisa de planos e modelos antes do código, porque mudar uma parede no papel custa muito menos do que derrubá-la.

Um modelo de processo é uma descrição simplificada de como equipes reais trabalham.



> **Pense assim.** Pense em uma aula de culinária. Cascata é seguir uma receita do início ao fim sem provar. O trabalho iterativo é provar o molho depois de cada passo e ajustar o próximo.

Um requisito precisa ser verificável: alguém deve poder dizer "atendido" ou "não atendido".



Erros encontrados aqui são os mais baratos de corrigir.

> **Pense assim.** Um alfaiate mede duas vezes e corta uma. Os requisitos são as medidas: se estiverem errados, nenhuma costura cuidadosa (programação) fará o terno servir.

Um modelo é útil quando é mais simples que o sistema, mas continua correto no que importa.



Pense assim. O mapa de uma cidade é um modelo. Ele ignora a cor de cada casa (abstrato), mas acerta as ruas (preciso), então você pode planejar uma rota antes de caminhar (preditivo e barato).

Os itens de configuração incluem tudo o que compõe o produto, não apenas o código.

- A GCS responde: qual versão, qual mudança, aprovada por quem e em qual linha de base.

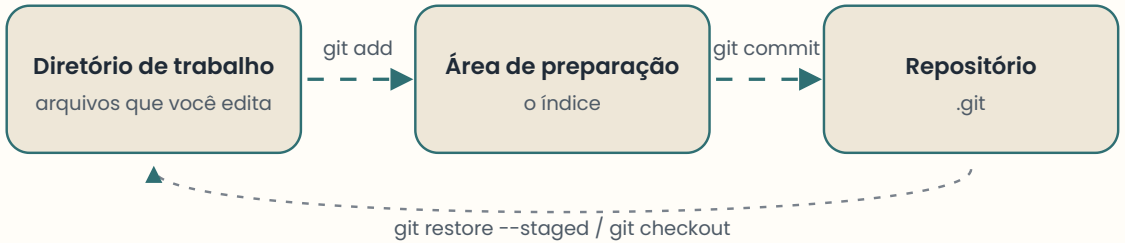
> **Pense assim.** Uma biblioteca mantém um catálogo de cada edição e de quem a emprestou. A GCS faz o mesmo com um produto: cada item, versão e mudança tem um registro que você pode consultar.

Um VCS guarda o histórico, permite recuperar versões e mostra quem mudou o quê e quando.

- O Git é distribuído; o GitHub é um serviço de hospedagem de repositórios Git.
- Um commit deve conter uma única mudança lógica com uma mensagem clara.

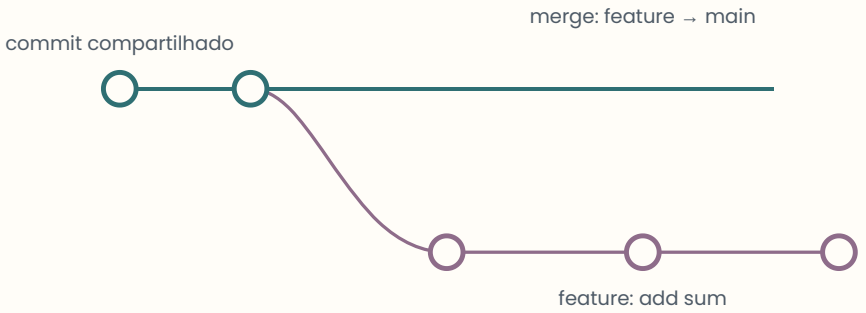
> **Pense assim.** O controle de versões é como um videogame com pontos de salvamento. Você pode voltar a qualquer ponto salvo, ver o que mudou entre dois deles e deixar que seus amigos joguem a partir da própria cópia.

Edite no diretório de trabalho, prepare com git add e registre com git commit.



> **Pense assim.** O Git é um estúdio fotográfico. O diretório de trabalho é o cenário, a preparação é escolher quais fotos vão para o álbum e o commit é imprimir a página do álbum. Uma branch é um segundo álbum que você pode montar sem mexer no primeiro.

origin é o nome convencional do remoto que você clonou ou conectou.



Cada ponto é um commit. Branches são ponteiros; o merge combina históricos.

> **Pense assim.** Um pull request é como um rascunho de redação enviado a um colega antes da entrega. O revisor não escreve por você, mas encontra erros enquanto ainda são baratos de corrigir.

Passo a passo

Coloque o ciclo de vida em ordem

1. Análise de requisitos
2. Design
3. Implementação
4. Testes
5. Implantação
6. Manutenção

Teste-se

1. Qual item é um item de configuração na gestão de configuração de software?

A) Um documento de requisitos B) A cafeteira do escritório C) Uma agenda pessoal

2. Qual afirmação sobre o Git está correta?

A) O Git é um sistema de controle de versões distribuído; cada clone contém o histórico completo. B) O Git não consegue registrar um commit sem uma conexão de internet com o GitHub. C) Git e GitHub são dois nomes para a mesma ferramenta.

3. Em um processo iterativo e incremental, o que representa a diferença entre duas linhas de base sucessivas?

A) Um incremento B) Um registro de riscos C) Apenas a entrega final

As respostas estão nas notas do apresentador.

Pratique depois

Laboratórios

- Inicialize um repositório e faça seu primeiro commit
- Leia o histórico e trabalhe em uma branch
- Resolva um conflito de merge
- Conecte um remoto e simule uma revisão

No site

- Simulador de Git